

uCore 构建系统感悟

前言

在完成 uCore 实验的过程中，我逐渐意识到：**内核代码只是冰山一角，真正让这个操作系统能够运行起来的，是一套精心设计的构建系统。**

本文记录我对 uCore 构建系统三个核心组件的理解与感悟：

- 双层 Makefile 机制**：主目录与 user 目录的协作
- user 文件夹的模式设置**：如何选择性编译测试程序
- C 代码到二进制的转换流程**：CMake + Python + objcopy 的工具链

一、双层 Makefile 机制

1.1 为什么需要两个 Makefile?

刚开始做实验时，我只关注 `make run` 能不能跑起来。后来遇到问题需要调试时，才发现这个项目有两个 Makefile：

```
uCore-Tutorial-Code-2025S-chX/
├─ Makefile          ← 主 Makefile (编译内核)
└─ user/
    └─ Makefile      ← 用户态 Makefile (编译用户程序)
```

为什么要分开?

通过阅读代码，我理解到这是一个**职责分离**的设计：

Makefile	职责	使用的工具链
主目录	编译内核代码、链接、生成镜像、运行 QEMU	<code>riscv64-unknown-elf-gcc</code> (裸机工具链)
user/	编译用户态程序、生成二进制文件	<code>riscv64-linux-musl-gcc</code> (musl libc 工具链)

关键洞察：内核和用户程序使用**不同的工具链**！

- 内核是裸机程序，不依赖任何标准库，使用 `elf-gcc`
- 用户程序需要 C 库支持 (printf、malloc 等)，使用 `musl-gcc`

1.2 主 Makefile 的核心逻辑

```
# 工具链定义
TOOLPREFIX = riscv64-unknown-elf-
CC = $(TOOLPREFIX)gcc
LD = $(TOOLPREFIX)ld
```

```
# 编译内核
build/kernel: $(OBJS) os/kernel.ld
    $(LD) $(LDFLAGS) -T os/kernel.ld -o $(BUILDDIR)/kernel $(OBJS)

# 调用 user/ 目录的 Makefile
user:
    make -C user CHAPTER=$(CHAPTER) BASE=$(BASE)

# 运行 QEMU
run: build/kernel $(F)/fs-copy.img
    $(QEMU) $(QEMUOPTS)
```

理解要点:

1. `make user` 会进入 `user/` 目录执行编译, 并传递 `CHAPTER` 和 `BASE` 参数
2. `make run` 依赖两样东西: 内核 (`build/kernel`) 和文件系统镜像 (`fs-copy.img`)
3. 文件系统镜像包含了用户程序的二进制文件

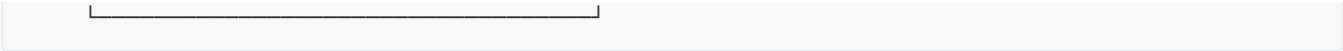
1.3 两个 Makefile 的协作流程

用户执行: `make user CHAPTER=8 BASE=1`



用户执行: `make run`





感悟：这种分层设计让每个 Makefile 只关注自己的职责，大大降低了复杂度。如果把所有逻辑都放在一个 Makefile 里，会变得难以维护。

二、user 文件夹的模式设置

2.1 CHAPTER 和 BASE 参数的作用

在做实验时，我经常使用这样的命令：

```
make user CHAPTER=8 BASE=1
make user CHAPTER=8
```

`BASE=1` 和不带 `BASE` 有什么区别？通过阅读 `user/Makefile`，我找到了答案：

```
CH8_BASE_TESTS := ch8b_ $(CH7_BASE_TESTS)
CH8_TESTS := $(CH8_BASE_TESTS) ch8_

ifeq ($(BASE), 1)
    CH8_TESTS := $(CH8_BASE_TESTS)
endif
```

参数	编译的测试程序	用途
<code>BASE=1</code>	只有 <code>ch8b_*</code> （基础测试）	验证框架代码是否正常
不带 <code>BASE</code>	<code>ch8b_*</code> + <code>ch8_*</code> （全部测试）	验证编程作业是否正确

理解：

- `ch8b_*` 是清华提供的基础测试，不需要编程作业就能通过
- `ch8_*` 是编程作业的测试，需要实现相应的系统调用才能通过

2.2 测试程序的累积继承

一个有趣的设计是：每章的测试会**累积前面所有章节的测试**：

```
CH2_BASE_TESTS := ch2b_
CH3_BASE_TESTS := ch3b_ $(CH2_BASE_TESTS)
CH4_BASE_TESTS := $(CH2_BASE_TESTS) ch3b_yield
CH5_BASE_TESTS := ch5b_ $(CH3_BASE_TESTS) usershell
CH6_BASE_TESTS := ch6b_ $(CH5_BASE_TESTS)
CH7_BASE_TESTS := ch7b_ $(CH6_BASE_TESTS)
CH8_BASE_TESTS := ch8b_ $(CH7_BASE_TESTS)
```

展开后：

- `CH8_BASE_TESTS` = `ch8b_ ch7b_ ch6b_ ch5b_ ch3b_ ch2b_ usershell`

为什么这样设计？

- 1. 回归测试：确保新功能不会破坏旧功能
- 2. 增量开发：每章只需要关注新增的测试
- 3. 完整验证：运行 ch8 测试时，会同时验证 ch2-ch7 的功能

2.3 测试程序的命名规范

命名模式	含义	示例
chXb_*	基础测试 (Base)	ch3b_sleep, ch5b_exit
chX_*	编程作业测试	ch3_trace, ch4_mmap0
chXt_*	特殊测试 (如调度算法)	ch5t_usertest
usershell	交互式 Shell	从 ch5 开始使用

感悟：这套命名规范让我一眼就能知道一个测试的性质。刚开始不理解为什么运行 ch8b_usertest 和 ch8_usertest 结果不一样，现在明白了——它们测试的内容范围不同。

三、C 代码到二进制的转换流程

3.1 整体流程概览

用户程序从源代码到最终被内核加载，经历了这样的转换过程：



3.2 CMakeLists.txt 的核心逻辑

`user/CMakeLists.txt` 是用户程序编译的核心配置：

```
# 工具链配置
set(PREFIX ${ARCH}-linux-musl-)
set(CMAKE_C_COMPILER ${PREFIX}gcc)

# 编译选项：裸机风格
set(CMAKE_C_FLAGS "${CMAKE_C_FLAGS} -fno-builtin -nostdinc -fno-stack-protector")
set(CMAKE_C_LINK_FLAGS "${LINK_FLAGS} -nostdlib -T ${ARCH_DIR}/user.ld")

# 遍历 src/ 目录，为每个 .c 文件生成可执行文件
foreach(PATH ${SRCS})
    get_filename_component(NAME ${PATH} NAME_WE)
    add_executable(${NAME} ${PATH})
    target_link_libraries(${NAME} ulib)

    # 生成反汇编文件
    add_custom_command(
        TARGET ${NAME} POST_BUILD
        COMMAND ${CMAKE_OBJDUMP} -d -S ${TARGET_FILE:${NAME}} > ${ASM_DIR}/${NAME}.asm
    )

    # 生成纯二进制文件
    add_custom_command(
        TARGET ${NAME} POST_BUILD
        COMMAND ${CMAKE_OBJCOPY} -O binary ${TARGET_FILE:${NAME}} ${BIN_DIR}/${NAME}
    )
endforeach()
```

理解要点：

1. 为什么用 `-nostdlib` ?
 - 用户程序不能使用标准 C 库，因为内核没有实现完整的系统调用
 - 而是使用 uCore 自己的 `ulib` 库
2. 为什么需要 `objcopy -O binary` ?
 - ELF 文件包含很多元数据（段表、符号表等）
 - 内核加载器只需要纯粹的二进制代码
 - `objcopy -O binary` 去掉所有元数据，只保留代码和数据
3. 为什么生成反汇编文件？
 - 调试时可以对照汇编代码分析问题
 - 特别是遇到页错误时，可以根据地址找到对应的代码

3.3 syscall_ids.h 的生成机制

一个容易踩坑的地方是系统调用号的定义：

```
add_custom_command(  
    OUTPUT syscall_ids.h  
    COMMAND sed -n -e s/___NR_/SYS_/p  
    < ${CMAKE_SOURCE_DIR}/${ARCH_DIR}/syscall_ids.h.in  
    > ${CMAKE_SOURCE_DIR}/lib/syscall_ids.h  
)
```

转换规则：

- 源文件：lib/arch/riscv/syscall_ids.h.in
- 目标文件：lib/syscall_ids.h（自动生成，不要手动修改！）
- 转换：___NR_read 63 → SYS_read 63

这个设计的意义：

- .in 文件使用 Linux 标准的 ___NR_ 前缀
- 用户程序使用 SYS_ 前缀（更直观）
- 通过 sed 自动转换，避免手动维护两份定义

踩坑经历：我曾经直接修改 syscall_ids.h，结果下次 make 时被覆盖了。后来才明白要修改 .in 文件。

3.4 Python 脚本：用户程序嵌入内核的关键

在 ch3-ch4 阶段（没有文件系统之前），用户程序是**直接嵌入到内核镜像中的**。这个过程由两个 Python 脚本完成：

```
scripts/  
├─ pack.py      ← 生成 link_app.S（嵌入用户程序二进制）  
└─ kernelld.py ← 生成 kernel_app.ld（动态链接脚本）
```

3.4.1 pack.py：将用户程序嵌入内核

pack.py 的作用是生成一个汇编文件 link_app.S，将所有用户程序的二进制文件**嵌入到内核的数据段中**。

```
# scripts/pack.py  
TARGET_DIR = "./user/target/bin/"  
  
f = open("os/link_app.S", mode="w")  
apps = os.listdir(TARGET_DIR)  
apps.sort()  
  
# 1. 写入应用程序数量  
f.write(''      .align 4  
      .section .data  
      .global _app_num  
_app_num:  
      .quad {}  
'' .format(len(apps)))
```

```

# 2. 写入每个应用的起始地址
for (idx, _) in enumerate(apps):
    f.write('    .quad app_{}_start\n'.format(idx))
f.write('    .quad app_{}_end\n'.format(len(apps) - 1))

# 3. 写入应用程序名称表
f.write('''
    .global _app_names
_app_names:
''')
for app in apps:
    f.write('    .string "' + app + '"\n')

# 4. 使用 .incbin 嵌入每个应用的二进制内容
for (idx, app) in enumerate(apps):
    f.write('''
        .section .data.app{0}
        .global app_{0}_start
app_{0}_start:
        .incbin "{1}"
    '''.format(idx, TARGET_DIR + app))
f.write('app_{}_end:\n'.format(len(apps) - 1))

```

生成的 link_app.S 示例:

假设有 3 个用户程序: ch2b_exit、ch2b_hello_world、ch2b_power, 生成的汇编如下:

```

.align 4
.section .data
.global _app_num
_app_num:
    .quad 3                # 应用程序数量

    .quad app_0_start      # ch2b_exit 的起始地址
    .quad app_1_start      # ch2b_hello_world 的起始地址
    .quad app_2_start      # ch2b_power 的起始地址
    .quad app_2_end        # 最后一个应用的结束地址

.global _app_names
_app_names:
    .string "ch2b_exit"    # 应用程序名称表
    .string "ch2b_hello_world"
    .string "ch2b_power"

.section .data.app0
.global app_0_start
app_0_start:
    .incbin "./user/target/bin/ch2b_exit"    # 嵌入二进制内容

.section .data.app1
.global app_1_start
app_1_start:

```

```

.incbn "./user/target/bin/ch2b_hello_world"

.section .data.app2
.global app_2_start
app_2_start:
.incbn "./user/target/bin/ch2b_power"
app_2_end:

```

关键理解：

1. **.incbin 指令**：这是汇编器的指令，作用是"原封不动地嵌入一个二进制文件"。这样用户程序的机器码就成为内核数据段的一部分。
2. **名称表的作用**：内核可以通过 `_app_names` 找到程序名，再通过 `app_x_start` 找到对应的二进制位置，实现"按名称加载程序"。
3. **为什么每个应用放在独立的 section**：`(.data.app0, .data.app1, ...)`——这是为了配合 `kernelld.py` 生成的链接脚本，确保每个应用按页对齐。

3.4.2 kernelld.py：生成动态链接脚本

`kernelld.py` 的作用是生成内核的链接脚本 `kernel_app.ld`。它需要根据用户程序的数量，动态生成数据段的布局。

```

# scripts/kernelld.py
TARGET_DIR = "./user/target/bin/"

f = open("os/kernel_app.ld", mode="w")
apps = os.listdir(TARGET_DIR)

# 写入链接脚本头部
f.write('''OUTPUT_ARCH(riscv)
ENTRY(_entry)
BASE_ADDRESS = 0x80200000;

SECTIONS
{
    . = BASE_ADDRESS;
    skernel = .;

    s_text = .;
    .text : {
        *(.text.entry)
        *(.text .text.*)
        . = ALIGN(0x1000);
        *(trampsec)
        . = ALIGN(0x1000);
    }

    . = ALIGN(4K);
    e_text = .;
    s_rodata = .;
    .rodata : {
        *(.rodata .rodata.*)

```



```

    }

    . = ALIGN(4K);
    e_rodata = .;
    s_data = .;
    .data : {
        *(.data)
    }
'''

# 关键：为每个应用程序的 section 添加对齐规则
for (idx, _) in enumerate(apps):
    f.write('        . = ALIGN(0x1000);\n')      # 按页对齐（4KB）
    f.write('        *(.data.app{})\n'.format(idx))

f.write('''
        . = ALIGN(0x1000);
        *(.data.*)
        *(.sdata .sdata.*)
    }

    . = ALIGN(4K);
    e_data = .;
    .bss : {
        *(.bss.stack)
        s_bss = .;
        *(.bss .bss.*)
        *(.sbss .sbss.*)
    }

    . = ALIGN(4K);
    e_bss = .;
    ekernel = .;

    /DISCARD/ : {
        *(.eh_frame)
    }
}
''')
```

为什么需要动态生成链接脚本？

普通程序的链接脚本是静态的，但 uCore 的链接脚本需要根据用户程序数量变化：

用户程序数量	需要的 section
3 个	<code>.data.app0</code> , <code>.data.app1</code> , <code>.data.app2</code>
5 个	<code>.data.app0</code> ~ <code>.data.app4</code>
10 个	<code>.data.app0</code> ~ <code>.data.app9</code>

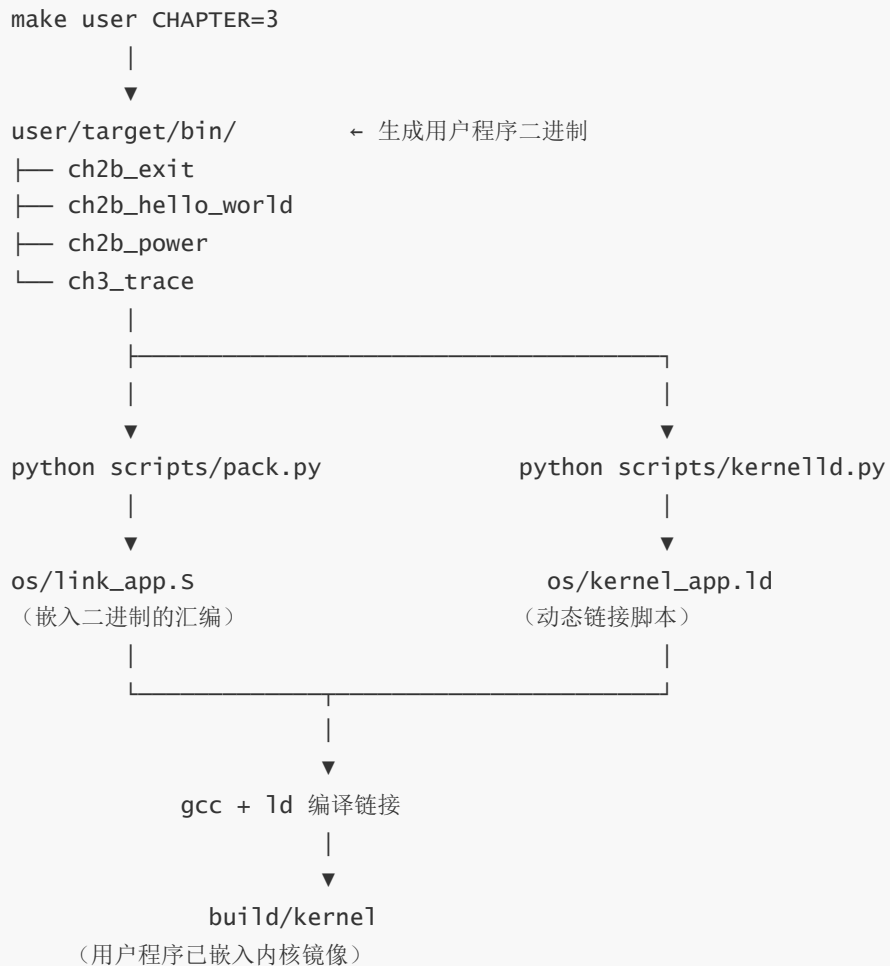
如果使用静态链接脚本，就需要预先定义足够多的 section，既不优雅也不灵活。

页对齐的原因：

```
. = ALIGN(0x1000);    // 4KB = 一页
*(.data.app0)
```

每个应用程序按页对齐，是为了方便内核加载时直接按页复制到用户地址空间。

3.4.3 两个脚本的协作流程



Makefile 中的调用：

```
os/link_app.o: $K/link_app.S
os/link_app.S: scripts/pack.py
    @$(PY) scripts/pack.py

os/kernel_app.ld: scripts/kernelld.py
    @$(PY) scripts/kernelld.py

build/kernel: $(OBSJS) os/kernel_app.ld
    $(LD) $(LDFLAGS) -T os/kernel_app.ld -o $(BUILDDIR)/kernel $(OBSJS)
```

3.4.4 为什么 ch5+ 不需要这两个脚本？

从 ch5 开始，uCore 引入了**文件系统**。用户程序不再嵌入内核，而是存储在文件系统镜像（`fs.img`）中：

阶段	用户程序存储方式	使用的脚本
ch2-ch4	嵌入内核镜像	<code>pack.py</code> + <code>kernelld.py</code>
ch5+	文件系统镜像	<code>nfs/fs.c</code> （打包工具）

这就是为什么 ch5+ 的 Makefile 不再调用 `pack.py` 和 `kernelld.py`。

3.4.5 initproc.py：指定初始进程名

从 ch5 开始，还有一个小脚本 `initproc.py`：

```
# scripts/initproc.py
parser = argparse.ArgumentParser()
parser.add_argument('INIT_PROC', default="usershell")
args = parser.parse_args()
f = open("os/initproc.S", mode="w")
f.write('''
    .global INIT_PROC
INIT_PROC:
    .string "{0}"
'''.format(args.INIT_PROC))
```

作用：生成一个包含初始进程名称的汇编文件。内核启动后会加载这个名字对应的程序（默认是 `usershell`）。

3.4.6 本项目的改动

在本项目（os-btbu）中，我们将 `pack.py` 和 `kernelld.py` 的功能整合到了一个脚本 `os/kernelld.py` 中，简化了构建流程。原理相同，只是减少了文件数量。

3.5 文件系统镜像的生成

`nfs/Makefile` 负责将用户程序打包成文件系统镜像：

```
$(FS_FUSE): fs.c fs.h types.h

fs.img: $(FS_FUSE)
    ./$(FS_FUSE) $$@ $(wildcard $(U)/$(USER_BIN_DIR)/*)
```

流程：

- 1. 编译 `fs.c` 生成 `fs` 工具
- 2. 运行 `./fs fs.img user/target/bin/*`
- 3. 该工具将所有二进制文件打包成一个简单的文件系统镜像

镜像格式：这是一个自定义的简单文件系统（不是 ext4 或 FAT），专门为 uCore 设计，包含：

- 超级块（superblock）

- 位图 (bitmap)
- inode 区域
- 数据区域

四、总结与感悟

4.1 构建系统是操作系统的"元系统"

在学习操作系统时，我们关注的是：进程管理、内存管理、文件系统...

但让这些代码能够运行起来的，是构建系统。没有 Makefile、CMake、链接脚本，再精妙的内核代码也只是一堆文本文件。

4.2 设计亮点总结

设计	亮点
双层 Makefile	职责分离，内核与用户程序独立编译
CHAPTER/BASE 参数	灵活控制测试范围，支持增量开发
测试累积继承	自动回归测试，确保向后兼容
.in 文件转换	源文件与生成文件分离，避免误修改
objcopy -O binary	去除 ELF 元数据，生成纯二进制
pack.py + kernelld.py	动态生成汇编和链接脚本，实现用户程序嵌入内核
.incbin 指令	汇编器直接嵌入二进制文件，无需手动转换
initproc.py	参数化配置初始进程，灵活切换启动程序

4.3 对后续开发的启示

1. **修改系统调用号时**：修改 `.in` 文件，而不是 `syscall_ids.h`
2. **调试用户程序时**：查看 `asm/` 目录下的反汇编文件
3. **只想测试基础功能时**：使用 `BASE=1` 参数
4. **想修改初始进程时**：使用 `INIT_PROC=xxx` 参数

4.4 感悟

"好的架构让复杂的事情变得简单，坏的架构让简单的事情变得复杂。"

uCore 的构建系统是一个很好的架构设计范例。它把"编译内核"、"编译用户程序"、"生成镜像"、"运行测试"这些复杂的任务，简化成了几个简单的 make 命令。

作为学习者，理解这套构建系统，不仅帮助我更顺利地完成任务，也让我学到了软件工程中"职责分离"、"约定优于配置"、"自动化优于手动"等重要原则。

附录：常用命令速查

```
# 编译用户程序（基础测试）
make user CHAPTER=8 BASE=1

# 编译用户程序（全部测试）
make user CHAPTER=8

# 编译内核并运行
make run

# 清理所有编译产物
make clean

# 只编译内核
make build

# 调试模式运行（需要 tmux + GDB）
make debug

# 查看生成的反汇编
cat user/asm/ch3_trace.asm
```